

Robótica e Automação - Trabalho Nº1

Leonardo Soares da Costa Tanaka - DRE: 121067652

Engenharia de Controle e Automação/UFRJ

Rio de Janeiro, Brasil

Novembro de 2024

1 Introdução

Este trabalho tem como objetivo explorar a modelagem cinemática de um sistema robótico composto por um braço robótico Kuka KR90 de 6 graus de liberdade e uma mesa posicionadora KUKA KP2 de 2 graus de liberdade. O efetuator do braço é uma tocha de soldagem, e a cinemática do sistema é descrita por meio de dois arquivos URDF: um para o braço e outro para a mesa. Com base nesses arquivos e na transformação homogênea T fornecida, será possível determinar os parâmetros de Denavit-Hartenberg (DH) de ambos os subsistemas (KR90 e KP2) e do sistema integrado KR90-KP2, visando descrever a cinemática direta e desenvolver o modelo no Matlab utilizando o toolbox de robótica de Peter Corke. O trabalho também inclui a validação da cinemática direta e inversa do sistema através de uma trajetória de waypoints pré-definidos, assim como a determinação e validação do Jacobiano do sistema.

2 Desenvolvimento

Foi considerado o sistema da figura composto por um braço robótico Kuka KR90 do tipo 6R e uma mesa posicionadora KUKA KP2 do tipo 2R.

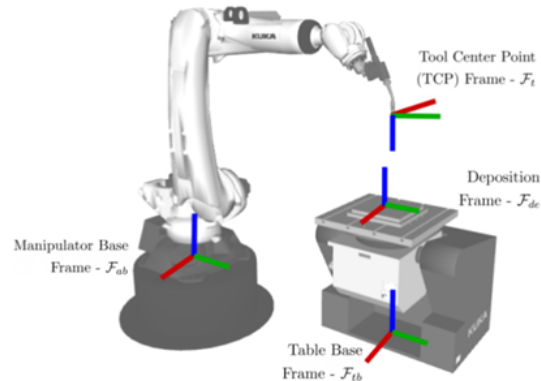


Figura 1: Braço robótico e mesa posicionadora

Foi considerado que o efetuador do braço manipulador é uma tocha para soldagem. Ademais, a cinemática deste sistema robótico é descrito em 2 arquivos urdf, um descrevendo a cinemática do KR90, kr90.urdf e outro descrevendo a cinemática da mesa posicionadora KP2, kp2.urdf. Além disso, foi considerado que a transformação homogênea do sistema de coordenadas da mesa F_{tb} com respeito a base do braço F_{ab} é dada por:

$$T_{at} = \begin{bmatrix} -1 & 0 & 0 & 1.73923 \\ 0 & -1 & 0 & 0.39198 \\ 0 & 0 & 1 & -0.46360 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (1)$$

2.1 Parâmetros de Denavit-Hartenberg do braço robótico KR90

Para determinar os parâmetros de Denavit-Hartenberg do braço robótico KR90, foi necessário analisar o URDF desse braço robótico que pode ser descrito pela seguinte tabela:

Joint i	parent	child	xyz	rpy	type	axis
1	kr90_base_link	kr90_link_1	0, 0, 0.675	0, 0, 0	revolute	0, 0, -1
2	kr90_link_1	kr90_link_2	0.35, 0, 0	0, 0, 0	revolute	0, 1, 0
3	kr90_link_2	kr90_link_3	1.35, 0, 0	0, 0, 0	revolute	0, 1, 0
4	kr90_link_3	kr90_link_4	0, 0, -0.041	0, 0, 0	revolute	-1, 0, 0
5	kr90_link_4	kr90_link_5	1.2, 0, 0	0, 0, 0	revolute	0, 1, 0
6	kr90_link_5	kr90_link_6	0.215, 0, 0	0, 1.5708, 0	revolute	0, 0, -1
7	kr90_link_6	kr90_tool0	0.03046, 0.033, 0.43161	-0.2356, 0.8988, -0.0942	fixed	-

Tabela 1: URDF do braço robótico KR90

Além disso, foi posicionado o eixo z do sistema de coordenadas 0 (z_0) no mesmo sentido e direção do eixo de rotação da junta 1. Fazendo a seguinte transformação homogênea da base para o eixo de coordenadas 0:

$$T_{b0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (2)$$

Então, foi possível obter os parâmetros do Denavit-Hartenberg Standard a seguir:

Elo i	θ_i	d_i	a_i	α_i	type	offset
1	θ_1	-0.675	0.35	$\pi/2$	revolute	0
2	θ_2	0	1.35	0	revolute	0
3	θ_3	0	0.041	$-\pi/2$	revolute	$\pi/2$
4	θ_4	-1.2	0	$\pi/2$	revolute	0
5	θ_5	0	0	$-\pi/2$	revolute	0
6	θ_6	-0.215	0	$-\pi$	revolute	0

Tabela 2: DH do braço robótico

Além disso, foi necessário obter a transformação homogênea do sistema de coordenadas 6 para o efetuador a seguir:

$$T_{6e} = \begin{bmatrix} 0.61978941 & -0.09040281 & 0.77954372 & 0.03046 \\ -0.05855747 & 0.98524594 & 0.16081497 & 0.033 \\ -0.78258041 & -0.14531952 & 0.60535125 & 0.43161 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (3)$$

Para obter os parâmetros de Denavit-Hartenberg e as transformações homogêneas acima, foi seguido os seguintes diagramas:

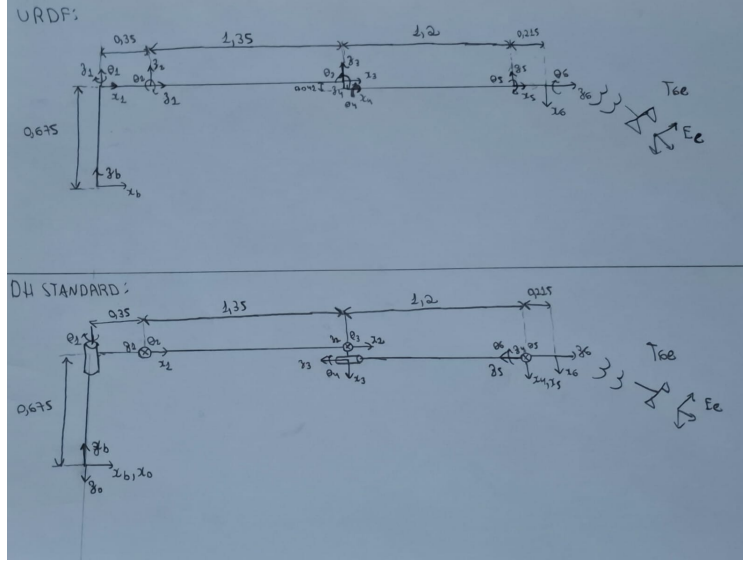


Figura 2: Diagramas do URDF e DH Standard do braço robótico

2.2 Parâmetros de Denavit-Hartenberg da mesa posicionadora KP2

Para determinar os parâmetros de Denavit-Hartenberg da mesa posicionadora KP2, foi necessário analisar o URDF dessa mesa posicionadora que pode ser descrita pela seguinte tabela:

Joint i	parent	child	xyz	rpy	type	axis
1	kp2_base_link	kp2_link_1	0, 0, 0.865	0, 3.1416, 0	revolute	1, 0, 0
2	kp2_link_1	kp2_link_2	0, 0, -0.22	0, 3.1416, 0	revolute	0, 0, 1
3	kp2_link_2	kp2_tool0	0, 0, 0	0, 0, 0	fixed	-

Tabela 3: URDF da mesa posicionadora KP2

Além disso, foi posicionado o eixo z do sistema de coordenadas 0 (z_0) no mesmo sentido e direção do eixo de rotação da junta 1. Fazendo a seguinte transformação homogênea da base para o eixo de coordenadas 0:

$$T_{b0} = \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0.865 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (4)$$

Então, foi possível obter os parâmetros do Denavit-Hartenberg Standard a seguir:

Elo i	θ_i	d_i	a_i	α_i	type	offset
1	θ_1	0	0	$-\pi/2$	revolute	$-\pi/2$
2	θ_2	0.22	0	0	revolute	$\pi/2$

Tabela 4: DH do braço robótico

Além disso, foi necessário obter a transformação homogênea do sistema de coordenadas 2 para o efetuador a seguir:

$$T_{2e} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (5)$$

Para obter os parâmetros de Denavit-Hartenberg e as transformações homogêneas acima, foi seguido os seguintes diagramas:

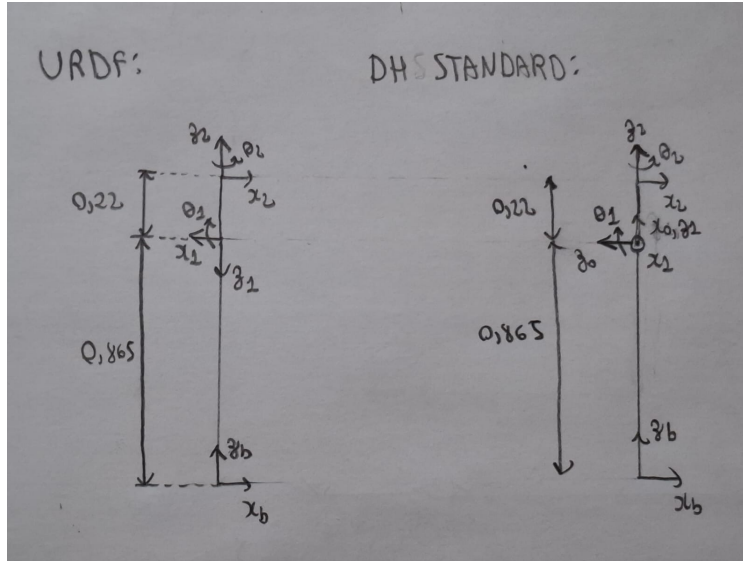


Figura 3: Diagramas do URDF e DH Standard da mesa posicionadora

2.3 Parâmetros de Denavit-Hartenberg dos sistemas KR90-KP2

Para determinar os parâmetros de Denavit-Hartenberg do sistema KR90-KP2, foi necessário analisar o URDF do braço robótico KP90, da mesa posicionadora KP2 e a transformação homogênea do sistema de coordenadas da mesa F_{tb} com respeito a base do braço F_{ab} , que estão respectivamente na Tabela 1, na Tabela 2 e na fórmula (1).

Além disso, foi posicionado o eixo z do sistema de coordenadas 0 (z_0) no mesmo sentido e direção do eixo de rotação da junta 1. Fazendo a seguinte transformação homogênea da base para o eixo de coordenadas 0:

$$T_{b0} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (6)$$

Então, foi possível obter os parâmetros do Denavit-Hartenberg Standard a seguir:

Elo i	θ_i	d_i	a_i	α_i	type	offset
1	θ_1	0.22	0	$\pi/2$	revolute	$\pi/2$
2	θ_2	1.73923	-0.39198	$-\pi/2$	revolute	0
3	θ_3	-0.27360	0.35	$\pi/2$	revolute	$\pi/2$
4	θ_4	0	1.35	0	revolute	0
5	θ_5	0	0.041	$-\pi/2$	revolute	$\pi/2$
6	θ_6	-1.2	0	$\pi/2$	revolute	0
7	θ_7	0	0	$-\pi/2$	revolute	0
8	θ_8	-0.215	0	$-\pi$	revolute	0

Tabela 5: DH do sistema KP90-KP2

Além disso, foi necessário obter a transformação homogênea do sistema de coordenadas 8 para o efetuador a seguir:

$$T_{8e} = \begin{bmatrix} 0.61978941 & -0.09040281 & 0.77954372 & 0.03046 \\ -0.05855747 & 0.98524594 & 0.16081497 & 0.033 \\ -0.78258041 & -0.14531952 & 0.60535125 & 0.43161 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad (7)$$

Para obter os parâmetros de Denavit-Hartenberg e as transformações homogêneas acima, foi seguido o seguinte diagrama:

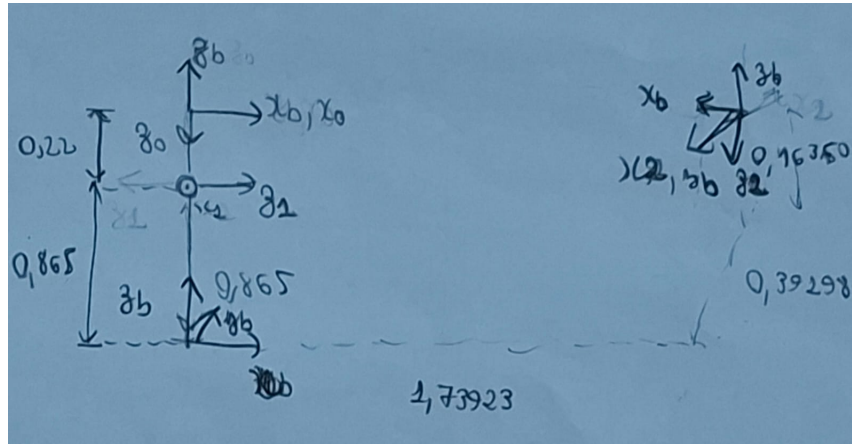


Figura 4: Diagrama do DH Standard do sistema KR90-KP2

2.4 Modelos dos sistemas

Utilizando o Robot Toolbox do Matlab foi construído os modelos utilizando a classe SerialLink de cada um dos 3 itens anteriores. Portanto, para realizar essa construção dos modelos, foi necessário utilizar a transformação homogênea da base para o eixo de coordenadas 0 (T_{b0}), o Denavit-Hartenberg do eixo de coordenadas 0 até o eixo de coordenadas do número de juntas (T_{0n}) e a transformação homogênea do último eixo de coordenadas do Denavit-Hartenberg até o eixo de coordenadas do efetuador (T_{ne}).

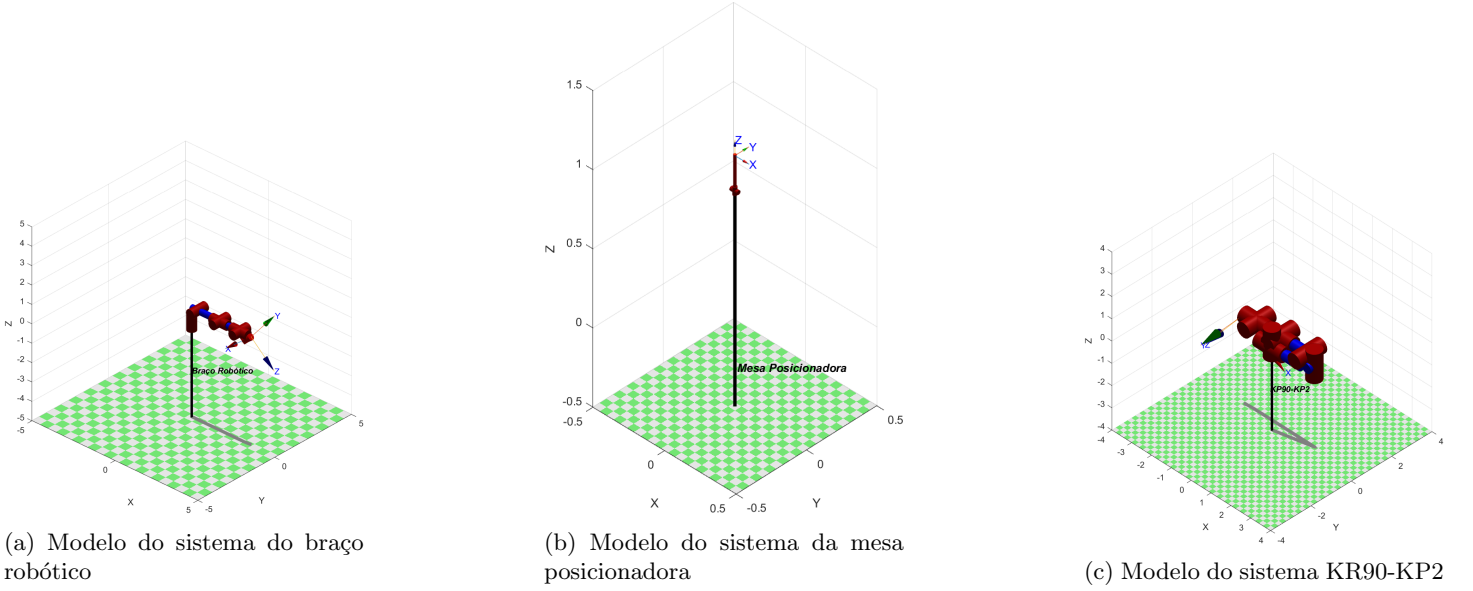


Figura 5: Modelos dos sistemas analisados. Cada subfigura apresenta um modelo específico utilizado no estudo.

2.5 Validação dos parâmetros de Denavit-Hartenberg

Foi feita a visualização de 4 configurações diferentes para cada modelo utilizando o Robot Toolbox do Matlab.

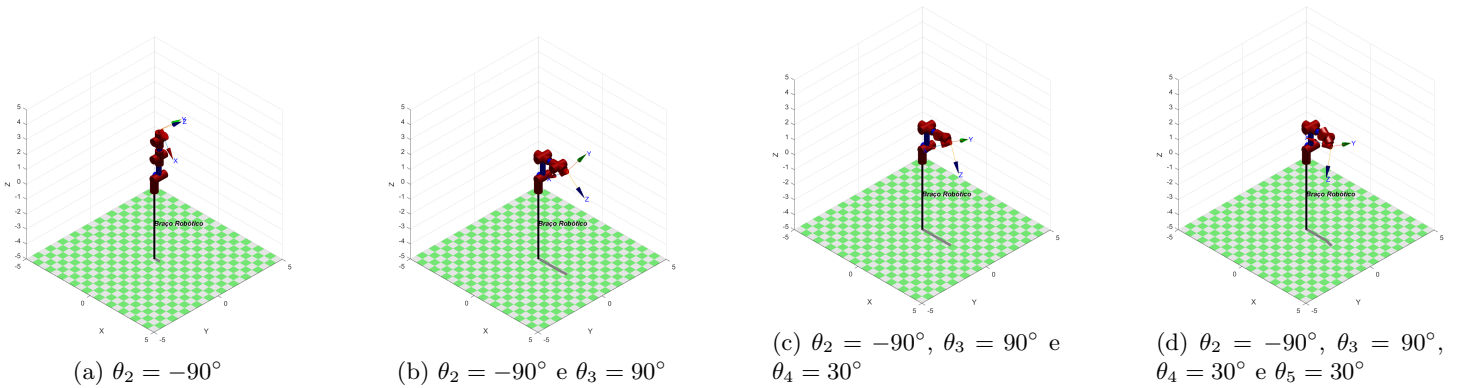


Figura 6: 4 configurações diferentes para o modelo do braço robótico.

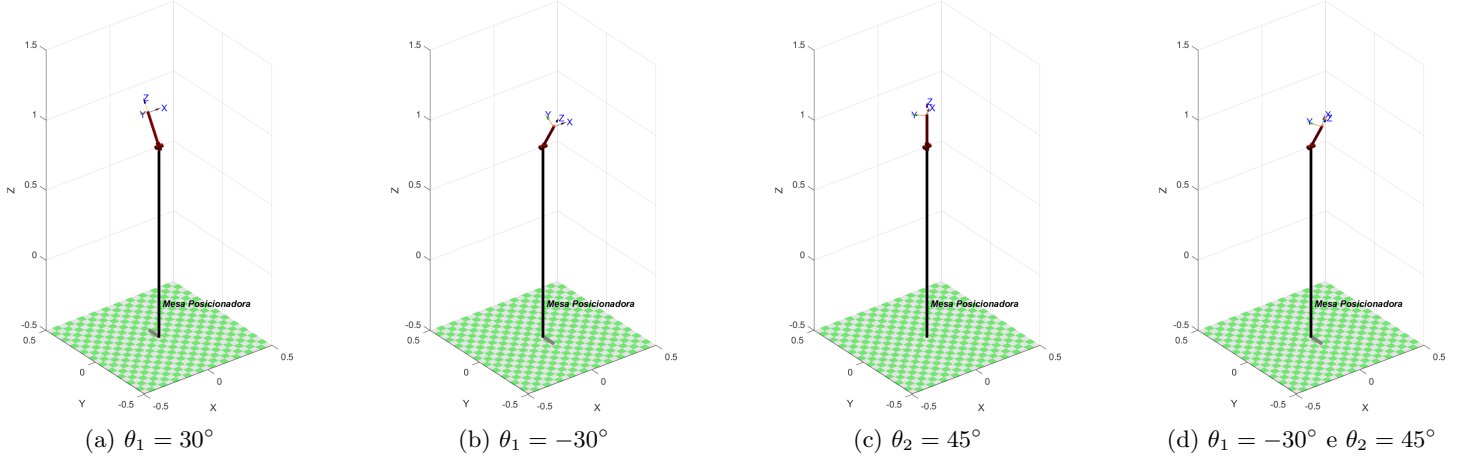


Figura 7: 4 configurações diferentes para o modelo da mesa posicionadora.

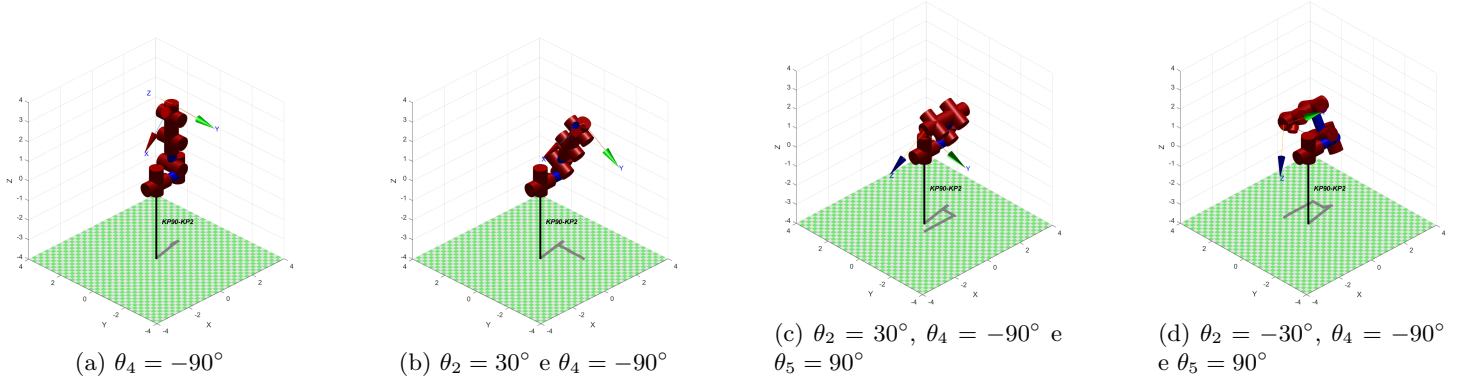


Figura 8: 4 configurações diferentes para o modelo do sistema KR90-KP2.

2.6 Verificação dos resultados da cinemática inversa

Neste estudo, foi explorado a função `ikine()` da *Robotics Toolbox* para calcular a cinemática inversa do robô serial Kr90, considerando uma trajetória em zigue-zague para o efetuador F_t em relação ao referencial F_{de} . Os *waypoints* utilizados são apresentados na Tabela 6. A orientação foi mantida constante para todos os casos, definida pelos ângulos de $roll = \pi$, $pitch = 0$ e $yaw = \pi/2$. Além disso, os ângulos das juntas da mesa posicionadora foram fixados em $\theta_t = [\pi/4, 0]$.

Waypoint	Posição p_{dt} (m)
1	$[0.04, 0, 0]^T$
2	$[-0.04, 0, 0]^T$
3	$[-0.04, 0.01, 0]^T$
4	$[0.04, 0.01, 0]^T$
5	$[0.04, 0.02, 0]^T$
6	$[-0.04, 0.02, 0]^T$
7	$[-0.04, 0.03, 0]^T$
8	$[0.04, 0.03, 0]^T$
9	$[0.04, 0.04, 0]^T$
10	$[-0.04, 0.04, 0]^T$

Tabela 6: *Waypoints* da trajetória em zigue-zague.

Os parâmetros da função `ikine()` foram explorados para cada *waypoint*, ajustando-se tolerância (`tol`), ângulos iniciais (`q0`), fator de amortecimento (`lambda`) e máscara de restrições (`mask`). Os resultados obtidos são apresentados a seguir:

Waypoint	θ_1	θ_2	θ_3	θ_4	θ_5	θ_6
1	0.2438	-0.7993	2.0019	1.6157	-1.7229	-0.2979
2	0.2329	-0.7770	1.9295	1.5979	-1.7205	-0.3488
3	0.2368	-0.7713	1.9271	1.6020	-1.7219	-0.3453
4	0.2479	-0.7933	1.9995	1.6200	-1.7241	-0.2942
5	0.2519	-0.7873	1.9971	1.6244	-1.7254	-0.2906
6	0.2406	-0.7655	1.9247	1.6061	-1.7234	-0.3419
7	0.2445	-0.7598	1.9222	1.6102	-1.7248	-0.3385
8	0.2560	-0.7813	1.9945	1.6287	-1.7266	-0.2870
9	0.2600	-0.7753	1.9919	1.6331	-1.7278	-0.2835
10	0.2484	-0.7540	1.9197	1.6142	-1.7262	-0.3352

Tabela 7: Valores dos ângulos das juntas (θ_1 a θ_6) para cada *waypoint*.

2.7 Determinação do Jacobiano de F_t com respeito a F_{de}

2.8 Validação do Jacobiano de F_t com respeito a F_{de}

2.9 Implementação da solução da cinemática inversa pelo método iterativo

3 Conclusão

Apêndice

Listing 1: Código do braço robótico (KR90)

```

1 % Parâmetros DH do robô
2 L1 = Revolute('d', -0.675, 'a', 0.35, 'alpha', pi/2, 'offset', 0);      % Elo 1
3 L2 = Revolute('d', 0, 'a', 1.35, 'alpha', 0, 'offset', 0);            % Elo 2
4 L3 = Revolute('d', 0, 'a', 0.041, 'alpha', -pi/2, 'offset', pi/2);     % Elo 3
5 L4 = Revolute('d', -1.2, 'a', 0, 'alpha', pi/2, 'offset', 0);          % Elo 4
6 L5 = Revolute('d', 0, 'a', 0, 'alpha', -pi/2, 'offset', 0);           % Elo 5
7 L6 = Revolute('d', -0.215, 'a', 0, 'alpha', -pi, 'offset', 0);        % Elo 6
8
9 % Definir o robô
10 robot = SerialLink([L1 L2 L3 L4 L5 L6], 'name', 'Braço Robótico');
11
12 % Transformação homogênea da base para o eixo 0 (T_b0)
13 T_b0 = [1, 0, 0, 0;
14         0, -1, 0, 0;
15         0, 0, -1, 0;
16         0, 0, 0, 1];
17
18 T_6e = transl(0.03046, 0.033, 0.43161) * trotx(-5.4) * troty(51.5) * trotx(-13.5);
19
20 % Incorporar as transformações no robô
21 robot.base = SE3(T_b0); % Define a base do robô
22 robot.tool = T_6e; % Define a ferramenta (tool) do robô
23
24 % Configuração inicial das juntas (todas em zero)
25 q0 = zeros(1, 6);
26
27 % Calcular a transformação homogênea do efetuador em relação à base (T_be)
28 T_be = robot.fkine(q0);
29
30 % Exibir a transformação final do efetuador
31 disp('Transformação do efetuador em relação à base (T_be):');
32 disp(T_be.T);
33
34 % Plotar o robô na configuração inicial com controles interativos
35 figure;
36 robot.teach(q0, 'workspace', [-5 5 -5 5 -5 5]);

```

Listing 2: Código da mesa poscionadora (KP2)

```

1 % Parâmetros DH do robô
2 L1 = Revolute('d', 0, 'a', 0, 'alpha', -pi/2, 'offset', -pi/2); % Elo 1
3 L2 = Revolute('d', 0.22, 'a', 0, 'alpha', 0, 'offset', pi/2); % Elo 2
4
5 % Definir o robô
6 robot = SerialLink([L1 L2], 'name', 'Mesa Posicionadora');
7
8 % Transformação homogênea da base para o eixo 0 (T_b0)
9 T_b0 = [0, 0, -1, 0;
10         0, 1, 0, 0;
11         1, 0, 0, 0.865;
12         0, 0, 0, 1];
13
14 % Transformação homogênea do eixo 2 para o efetuador (T_2e)
15 T_2e = eye(4); % Matriz identidade, conforme especificado
16
17 % Incorporar as transformações no robô
18 robot.base = SE3(T_b0); % Define a base do robô
19 robot.tool = SE3(T_2e); % Define a ferramenta (tool) do robô
20
21 % Configuração inicial das juntas (todas em zero)
22 q0 = zeros(1, 2);
23
24 % Calcular a transformação homogênea do efetuador em relação à base (T_be)
25 T_be = robot.fkine(q0);
26
27 % Exibir as transformações
28 disp('Transformação da base para o efetuador (T_be):');
29 disp(T_be.T);
30
31 % Plotar o robô na configuração inicial com controles interativos
32 figure;
33 robot.teach(q0, 'workspace', [-0.5 0.5 -0.5 0.5 -0.5 1.5]);

```

Listing 3: Código do sistema (KR90-KP2)

```

1 % Parâmetros DH do sistema KP90-KP2
2 L1 = Revolute('d', 0.22, 'a', 0, 'alpha', pi/2, 'offset', pi/2); % Elo 1
3 L2 = Revolute('d', 1.73923, 'a', -0.39198, 'alpha', -pi/2, 'offset', 0); % Elo 2
4 L3 = Revolute('d', -0.27360, 'a', 0.35, 'alpha', pi/2, 'offset', pi/2); % Elo 3
5 L4 = Revolute('d', 0, 'a', 1.35, 'alpha', 0, 'offset', 0); % Elo 4
6 L5 = Revolute('d', 0, 'a', 0.041, 'alpha', -pi/2, 'offset', pi/2); % Elo 5
7 L6 = Revolute('d', -1.2, 'a', 0, 'alpha', pi/2, 'offset', 0); % Elo 6
8 L7 = Revolute('d', 0, 'a', 0, 'alpha', -pi/2, 'offset', 0); % Elo 5
9 L8 = Revolute('d', -0.215, 'a', 0, 'alpha', -pi, 'offset', 0); % Elo 6
10
11 % Definir o robô
12 robot = SerialLink([L1 L2 L3 L4 L5 L6 L7 L8], 'name', 'KP90-KP2');
13
14 T_8e = transl(0.03046, 0.033, 0.43161) * trotx(-5.4) * troty(51.5) * trotx(-13.5);
15
16 % Incorporar as transformações no robô
17 robot.base = trotx(180); % Define a base do robô
18 robot.tool = T_8e; % Define a ferramenta (tool) do robô
19
20 % Configurar o estado inicial das juntas (todas em zero)
21 q0 = zeros(1, 8);
22
23 % Calcular a transformação homogênea do efetuador em relação à base (T_be)
24 T_be = robot.fkine(q0);
25
26 % Exibir as transformações
27 disp('Transformação da base para o efetuador (T_be):');
28 disp(T_be.T);
29
30 % Plotar o robô na configuração inicial com controles interativos
31 figure;
32 robot.teach(q0, 'workspace', [-4 4 -4 4 -4 4]);

```

Listing 4: Código da subseção 2.6

```

1 % Configuração dos Waypoints
2 waypoints = [ 0.04, 0.00, 0.0; -0.04, 0.00, 0.0; -0.04, 0.01, 0.0; 0.04, 0.01, 0.0; 0.04, 0.02, 0.0;
3             -0.04, 0.02, 0.0; -0.04, 0.03, 0.0; 0.04, 0.03, 0.0; 0.04, 0.04, 0.0; -0.04, 0.04, 0.0;];
4 % ----- Configuração do Robô Kuka KR90 -----
5 % ----- Configuração do Robô Kuka KP2 (Mesa Posicionadora) -----
6 % ----- Configuração da Transformação entre Robôs -----
7 % Transformação entre as bases dos robôs
8 T_Fab_Ftb = SE3;
9 T_Fab_Ftb.t = [1.73923, 0.39198, -0.46360]; % Translação
10 T_Fab_Ftb = T_Fab_Ftb * SE3.Rz(180); % Rotação de 180 graus em Z
11
12 % Angulos conhecidos da mesa posicionadora
13 theta_t = [pi/4, 0];
14 T_Ftb_Fde = robot_kukaKp2.fkine(theta_t); % Cinemática direta do KP2
15 % ----- Cálculo da Cinemática Inversa e Trajetoria -----
16 % Configurações iniciais
17 q0 = zeros(1, robot_kukaKr90.n); % Angulos iniciais das juntas
18 tol = 1e-6; % Tolerância para solução da cinemática inversa
19 lambda = 0.1; % Taxa de regularização
20
21 % Matriz para armazenar as soluções das juntas
22 q_solutions = zeros(size(waypoints, 1), robot_kukaKr90.n);
23
24 % Iteração para cada waypoint
25 for i = 1:size(waypoints, 1)
26     % Define a transformação desejada para o waypoint atual
27     T_Fde_Ft = SE3.Rz(90) * SE3.Rx(180); % Orientação fixa
28     T_Fde_Ft.t = waypoints(i, :); % Posição do waypoint
29
30     % Transformação total em relação a base absoluta
31     T_Fab_Ft = T_Fab_Ftb * T_Ftb_Fde * T_Fde_Ft;
32
33     % Calcula a cinemática inversa
34     q_solutions(i, :) = robot_kukaKr90.ikine(T_Fab_Ft, q0, 'tol', tol, 'lambda', lambda, 'mask', [1 1 1 1
35     1 1]);
36
37     % Valida a solução obtida com a cinemática direta
38     T_check = robot_kukaKr90.fkine(q_solutions(i, :));
39     disp(['Waypoint ', num2str(i)]);
40     disp('Transformação homogênea obtida:');
41     disp(robot_kukaKr90.fkine([0, pi/4, q_solutions(i,:) ]));
42     disp('Angulos das juntas:');
43     disp(q_solutions(i, :));
44 end

```